

Architecture Decision Record

System: legal precedent research agent over 56 Indian court judgments **Date:** August 2026

Design decisions only. Evaluation methodology, full results and the failure analysis live in `RESULTS.md`; every number quoted here is produced by `evals/` and reproducible with `make evals`.

1. Why this architecture

Three measured facts about the corpus drove every choice: it is *small* (56 judgments, 936 pages, ~692K tokens), it is *deliberately mixed* (only ~32 are motor-accident matters; the rest are trademark, excise, cheque dishonour — a planted precision trap), and the brief's own example query "which judgments involve commercial vehicles?" is **structured, not semantic** — it has an *enumerable* answer (8 by the adjudicated gold set, 17 by the enrichment layer's own flag; the gap is a definitional question, not a ranking one) that vector top-k structurally cannot express at any k.

Agent framework — LangGraph, as a cycle rather than a pipeline. The brief forbids hard-coded pipelines, so the agent is a tool-calling loop: it chooses among five tools every turn and decides its own next step. There is no `if query_type == ...` anywhere in the control flow. LangGraph over bare LangChain for explicit state and a routing layer I can unit-test; over CrewAI because it is disallowed, and over LlamaIndex because this is an agent problem, not an indexing problem.

Retrieval — hybrid, fused, reranked, resolved to documents. Qwen3-Embedding-0.6B (dense) + BM25 (sparse) → reciprocal rank fusion → LLM rerank. The embedding model was chosen on **MLEB**, the legal-domain benchmark: 77.13 nDCG@10 versus BGE-M3's 69.44 — a legal-specific measurement rather than a general MTEB score. LanceDB is the store because it is embedded: the brief forbids submissions needing local infrastructure, so a server-based vector DB was disqualified on the rules, not on preference.

The real centre of the design is offline enrichment. One LLM pass per judgment produces a structured `CaseCard` — holding, ratio, statutes, citations, and crucially `outcome_favours ∈ {claimant, insurer, mixed, neutral}`. This is what makes structured questions answerable *exactly* (`filter_judgments` scans all 56 and returns every match, not a top-k sample) and what makes adverse retrieval possible at all: "find what the other side will cite" is a metadata query, not a semantic one.

Chunking — structure-aware, with a deliberate split of duties. Judgments are split on their own section boundaries, each chunk prefixed with case/court/ section context so it stands alone. But **candidate generation runs over case cards, not chunks**: each judgment is embedded as one synthesised document of its legal substance (holding, ratio, issues, statutes, outcome). Chunks are retrieved only *within* an already-selected judgment, to supply verbatim quotes. Cards decide what is relevant; chunks prove it. In law the citable unit is the judgment, and Indian judgments carry enough recital boilerplate that chunk-level matching ranks furniture. The cost — one vector per judgment is lossy — is the first thing that breaks at 5,000 documents (§4 below).

2. Tradeoffs, and what they cost

- **Enrichment is a single point of failure.** Every downstream capability inherits its errors. Bought: exact enumeration and adverse surfacing.
- **Precision traded for adverse coverage.** Redefining "adverse" tripled adverse recall and cost ~10 points of precision. Deliberate: a reader can discount a weak citation but cannot discount authority they were never shown.
- **Cost is quadratic in tool calls** (~600·n², measured). The loop resends its history each turn. Three compaction variants were built and **all three failed measurement** (see RESULTS.md) — the honest state is that this is diagnosed, not solved.
- **Determinism sacrificed to model choice.** DeepSeek-V4-Flash is Mixture-of-Experts; identical prompts at temperature 0 return different text because expert routing depends on server-side batching. Verified directly.
- **A bespoke evaluation, not an off-the-shelf one.** The corpus is small enough to label exhaustively (840 labels), and Dimension 4 — adverse identification, the brief's stated thesis — has no equivalent in any framework. Cross-checked

against RAGAS rather than asserted: it corroborates retrieval (context-precision 1.000 on the research queries) and diverges exactly where its assumptions break, on enumerative answers and on legal argument that is *supposed* not to be in the source (see RESULTS.md).

3. How the agent decides simple vs. deep

It doesn't branch — it budgets. A triage call classifies the request as `simple` or `research` and sets a *starting step allowance* (4 vs 14). Both depths then run the **identical graph with identical tools**; only the allowance differs, and the agent can escalate its own budget mid-run if it is still finding new documents. An unrecognised label defaults to the deeper allowance, because under-researching is the costlier error.

Depth is therefore an emergent property of the agent's own tool choices, not a routing decision made for it. The output *shape* is likewise the agent's call: it elects `submit_research_report` or `submit_answer` as a tool.

Measured on the final run — triage's own label, the effort it actually spent, and the contract it chose:

query	triage said	tool calls	contract
q14_summarise	simple	2	DirectAnswer ✓
q02_commercial	simple	3	DirectAnswer ✓
q12_s166	simple	23	DirectAnswer ✓
q01_brief	research	30	ResearchReport ✓
q05_flip_insurer	research	32	ResearchReport ✓

A 16× spread in effort from one code path with no branching. `q12` shows the design working as intended: triage called it *simple*, the agent then escalated its own budget to 23 calls because it was still finding new judgments, and it still returned the short contract. The budget is a starting allowance, not a ceiling — as a gate it would have stopped at four and under-reported.

Measured: trajectory accuracy **100%**, contract accuracy **75%**. Every miss is over-delivery — a full report where a lookup would have served — and the answers were still correct, so the cost is tokens rather than accuracy.

4. What changes at 5,000 documents

Measured baseline: 56 judgments → 1,629 chunks, 13 MB. At 89× scale the index is ~1 GB — and **index size is not what breaks**. Three things do:

Architecture. Candidate generation currently matches one embedding *per judgment*. A judgment deciding five issues over forty pages becomes a single point in embedding space, so a query about its third issue never pulls it — at 56 documents it lands in the top 30 regardless, at 5,000 it is buried. Recall must move to **chunk-level ANN → roll up to parent judgments → card-level scoring of ~200 survivors → cascade rerank** (cheap cross-encoder, then the existing LLM reranker over ~20). The card layer stops being the recall substrate and becomes the filtering and reasoning substrate.

Code. `all_card_summaries()` feeds every card into one prompt — measured 4.4K tokens at 56, **≈392K at 5,000** — so `screen_corpus` becomes staged screening. `filter_cards(limit=200)` silently truncates: invisible at 56 where no filter can exceed the cap, quietly wrong at 5,000 on exactly the enumerative queries it exists to answer — **a correctness bug, and the first thing I would fix**. `build_index` needs an ANN index and content-hash incremental upsert rather than `reset=True`; enrichment needs append-only checkpoints and a schema/model version stamp.

Pipeline. Batch → incremental: content-hash idempotency, enrichment as a queue with dead-lettering, and corpus versions pinned to gold-set versions so eval numbers stay comparable as the corpus grows weekly.

Newly necessary: negative-treatment detection (citing an overruled judgment is negligence-grade), citation-graph authority, court hierarchy as a ranking prior, per-query cost as an SLO.

Survives unchanged: the trace design, the tool interface, the agent loop, the eval dimensions, the retrieved-vs-cited separation. **The agent layer is scale-invariant; everything that breaks is in the retrieval and data layers** — the right

place for it, since those sit behind stable tool signatures.

5. What I would change with another week

Ordered by expected value: **(1)** adverse *retrieval* breadth on insurer-side matters — categorisation is fixed and shipped, coverage is not; **(2)** sub-agent reads, so full judgment text never enters the loop's history — the structural answer to the cost curve, after mechanical compaction failed three times and Azure was confirmed not to expose prompt caching; **(3)** decouple quote verification from the revision budget; **(4)** negative-treatment detection — the system will currently cite an overruled judgment with full confidence; **(5)** a human-validated judge, since Dimension 3 rests on an unvalidated LLM.

Results, in brief

	tuned	held-out
precision / nDCG@10	73.0% / 0.740	53.0% / 0.576
retrieval recall	93.1%	100%
answer recall	82.5%	81.7%
reasoning rubric (independent judge)	67.1%	—
adverse recall	51.2%	33.3%
hallucinated citations	0	0
run success	21/21	—

Gold set: 840 labelled pairs, two annotators on different models, κ 0.748, of which **504 are verified by literal text search** rather than model judgement.

Twelve behavioural changes were measured one variable at a time; four survived. The largest single improvement was not a mechanism but a definition — redefining "adverse" from "*this defeats us*" to "*the opposing side will cite this*" moved adverse recall 0.181 → 0.648 and improved every held-out metric. Rejected candidates remain behind configuration switches with their measurements attached, so each decision is reproducible.

Evaluation framework, methodology, per-dimension results and failure analysis: [RESULTS.md](#).